

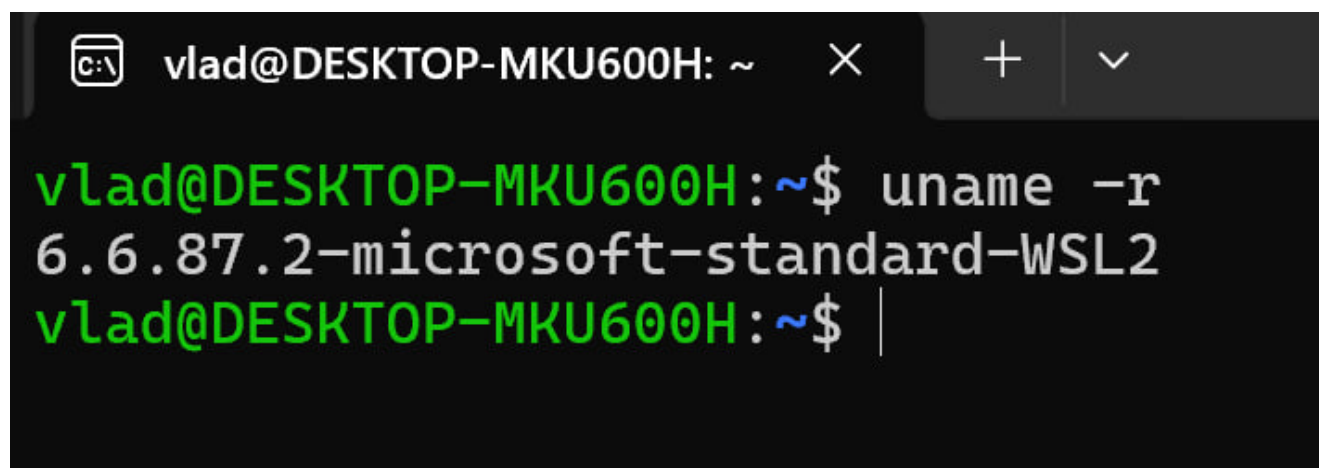
Не смотря на проблемы со сборкой ядра под wsl, приступим к этой задаче.

Для начала, хочу заметить, что под wsl это реализовать невозможно т.к. сама первая версия выглядит лишь визуальной копией Линукса (как я понял). А вот wsl2 реализует полноценную операционную систему (в базовом её понимании). Сложные, необычные библиотеки, модули, девайсы, могут быть еще не реализованы. Например, tun/tap появилось в wsl2 около 5-6 лет назад).

Приступим к задаче.

Для начала, проверим информацию о релизе системы:

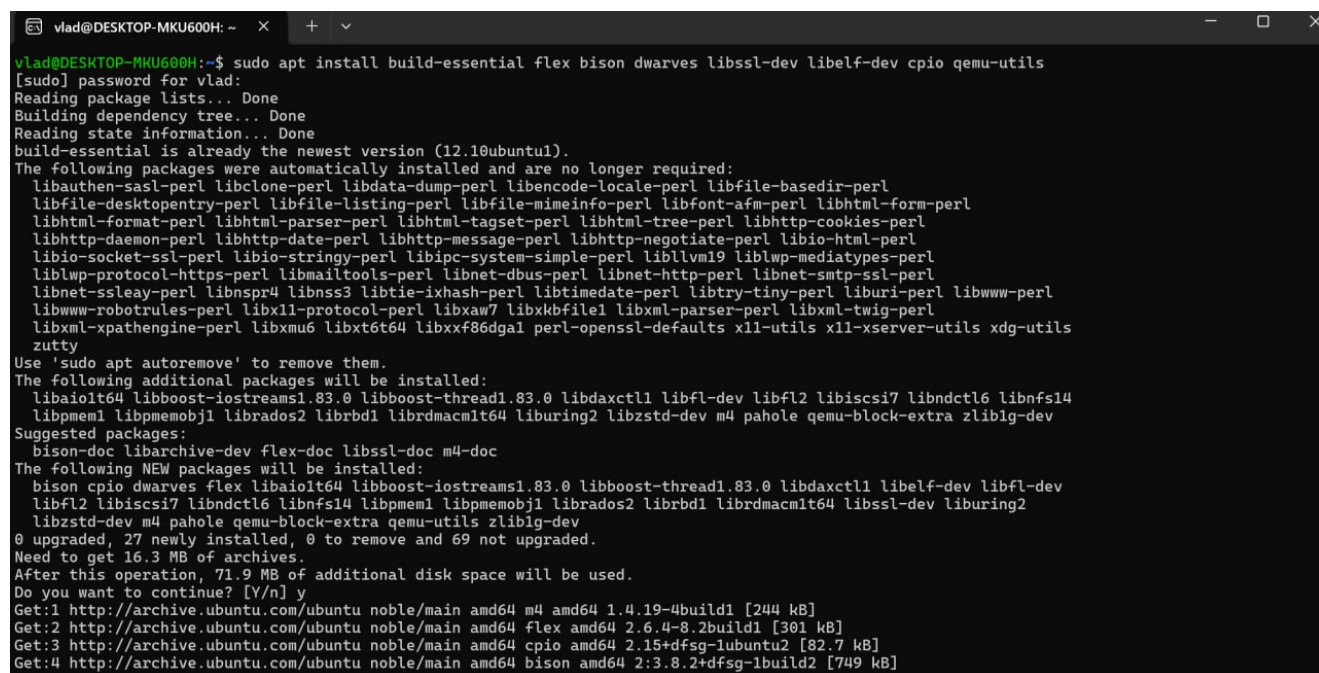
```
$ uname -r
```



```
vlad@DESKTOP-MKU600H: ~  
vlad@DESKTOP-MKU600H:~$ uname -r  
6.6.87.2-microsoft-standard-WSL2  
vlad@DESKTOP-MKU600H:~$
```

Дальше установим необходимые для нас модули:

```
$ sudo apt install build-essential flex bison dwarves libssl-dev libelf-dev cpio qemu-utils
```



```
vlad@DESKTOP-MKU600H:~$ sudo apt install build-essential flex bison dwarves libssl-dev libelf-dev cpio qemu-utils  
[sudo] password for vlad:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
build-essential is already the newest version (12.10ubuntu1).  
The following packages were automatically installed and are no longer required:  
  libauthen-sasl-perl libclone-perl libdata-dump-perl libencode-locale-perl libfile-basedir-perl  
  libfile-desktopentry-perl libfile-listing-perl libfile-mimeinfo-perl libfont-afm-perl libhtml-form-perl  
  libhtml-format-perl libhtml-parser-perl libhtml-tagset-perl libhtml-tree-perl libhttp-cookies-perl  
  libhttp-daemon-perl libhttp-date-perl libhttp-message-perl libhttp-negotiate-perl libio-html-perl  
  libio-socket-ssl-perl libio-stringy-perl libipc-system-simple-perl liblvm2-lsdevd-perl liblwp-mediatypes-perl  
  liblwp-protocol-https-perl libmailtools-perl libnet-dbus-perl libnet-http-perl libnet-smtp-ssl-perl  
  libnet-ssleay-perl libnss3 libnss3-dev libtie-ixhash-perl libtimedate-perl libtiny-perl liburi-perl libwww-perl  
  libwww-robotrules-perl libxml1-perl libxaw7 libxkbfile1 libxml-parser-perl libxml-twig-perl  
  libxml-xpathengine-perl libxmu6 libxt6t64 libxxf86dga1 perl openssl-defaults x11-utils x11-xserver-utils xdg-utils  
zutty  
Use 'sudo apt autoremove' to remove them.  
The following additional packages will be installed:  
  libaio1t64 libboost-iostreams1.83.0 libboost-thread1.83.0 libdaxctl1 libelf-dev libelf1 libiscsi7 libndctl6 libnfs14  
  libpmem1 libpmemobj1 librados2 librbd1 librdmacm1t64 liburing2 libzstd-dev m4 pahole qemu-block-extra zlib1g-dev  
Suggested packages:  
  bison-doc libarchive-dev flex-doc libssl-doc m4-doc  
The following NEW packages will be installed:  
  bison cpio dwarves flex libaio1t64 libboost-iostreams1.83.0 libboost-thread1.83.0 libdaxctl1 libelf-dev libelf1  
  libfl2 libiscsi7 libndctl6 libnfs14 libpmem1 libpmemobj1 librados2 librbd1 librdmacm1t64 libssl-dev liburing2  
  libzstd-dev m4 pahole qemu-block-extra qemu-utils zlib1g-dev  
0 upgraded, 27 newly installed, 0 to remove and 69 not upgraded.  
Need to get 16.3 MB of archives.  
After this operation, 71.9 MB of additional disk space will be used.  
Do you want to continue? [Y/n] y  
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 m4 amd64 1.4.19-4build1 [244 kB]  
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 flex amd64 2.6.4-8.2build1 [301 kB]  
Get:3 http://archive.ubuntu.com/ubuntu noble/main amd64 cpio amd64 2.15+dfsg-1ubuntu2 [82.7 kB]  
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 bison amd64 2:3.8.2+dfsg-1build2 [749 kB]
```

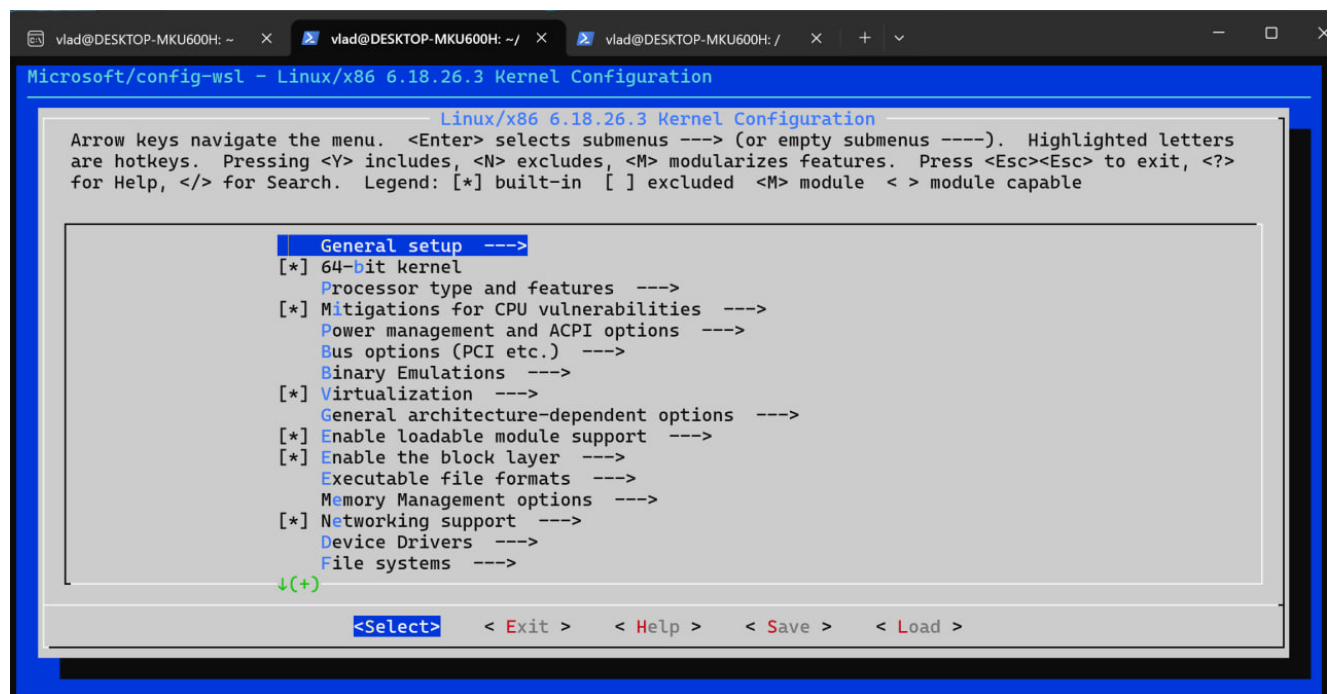
Затем нам нужно скачать само ядро Линукс, подходящее для нашей WSL2. Я выбрал официальное ядро от Microsoft:

```
$ git clone https://github.com/microsoft/WSL2-Linux-Kernel.git
```

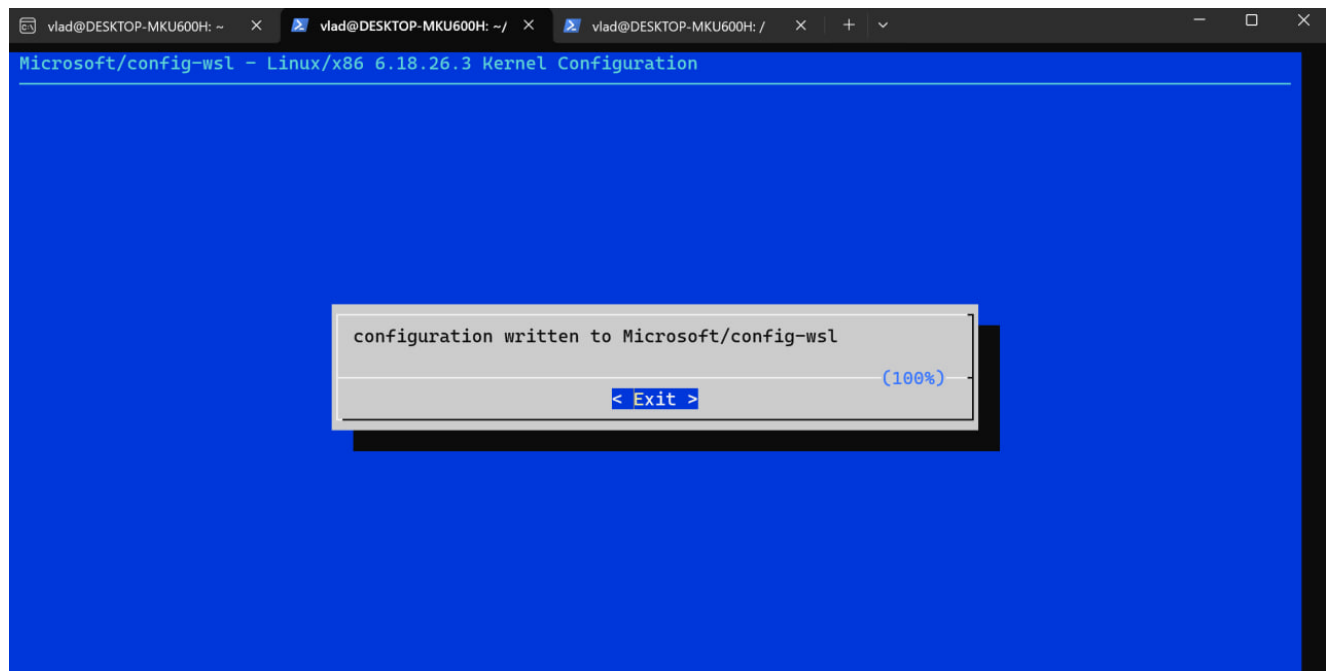
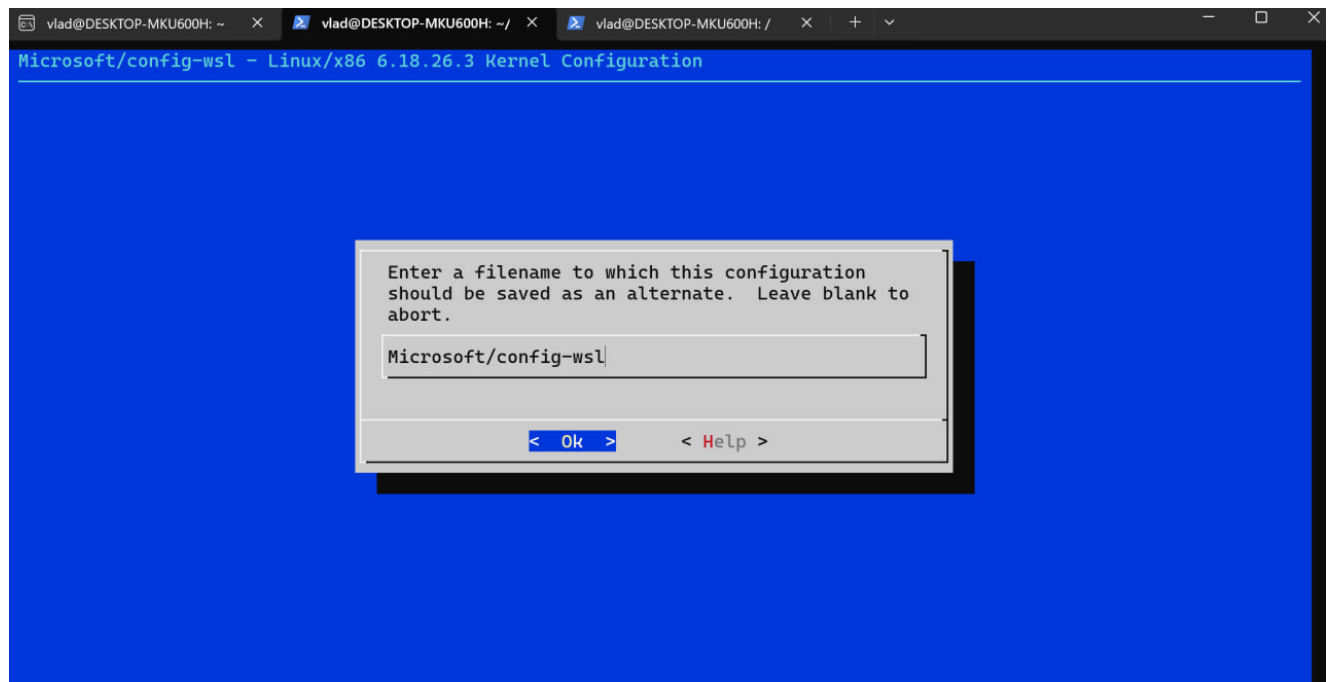
```
vlad@DESKTOP-MKU600H:~$ git clone https://github.com/microsoft/WSL2-Linux-Kernel.git
Cloning into 'WSL2-Linux-Kernel'...
remote: Enumerating objects: 12749461, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (13/13), done.
Receiving objects: 74% (9520785/12749461), 2.60 GiB | 3.49 MiB/s
```

Заходим в самую папку с ядром и начинаем его настраивать ядро. С помощью команды можно вызвать графическое меню с предустановленной конфигурацией от Microsoft и ускорить её выполнение, передав параллельное управление на все доступные ядра процессора:

```
$ make menuconfig KCONFIG_CONFIG=Microsoft/config-wsl -j$(nproc)
```



Так как сборка уже подходит под мою систему, то мне требуется только сохранить её.



```

vlad@DESKTOP-MKU600H:~/WSL2-Linux-Kernel$ make menuconfig KCONFIG_CONFIG=Microsoft/config-wsl -j$(nproc)
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/mnconf-common.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTCC scripts/kconfig/mconf.o
HOSTCC scripts/kconfig/lxdialog/checklist.o
HOSTCC scripts/kconfig/lxdialog/inputbox.o
HOSTCC scripts/kconfig/lxdialog/menubox.o
HOSTCC scripts/kconfig/lxdialog/textbox.o
HOSTCC scripts/kconfig/lxdialog/util.o
HOSTCC scripts/kconfig/lxdialog/yesno.o
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTLD scripts/kconfig/mconf

*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

vlad@DESKTOP-MKU600H:~/WSL2-Linux-Kernel$

```

Так как это wsl2, а не обычное ядро Linux, то нам нужно только скопировать ядро. В wsl2 нет grub, поэтому нам не нужно делать make install. А также нет /lib/modules, потому что они сразу встроены в vmlinux (результат компиляции команды ниже).

\$ make KCONFIG_CONFIG=Microsoft/config-wsl -j\$(nproc)

```

BTF [M] net/bluetooth/bnep/bnep.ko
LD [M] net/sctp/sctp_diag.ko
BTF [M] net/bluetooth/hidp/hidp.ko
LD [M] net/mac80211/mac80211.ko
LD [M] net/dns_resolver/dns_resolver.ko
LD [M] net/ceph/libceph.ko
BTF [M] net/sunrpc/auth_gss/rpcsec_gss_krb5.ko
LD [M] net/openvswitch/openvswitch.ko
LD [M] net/openvswitch/vport-vxlan.ko
BTF [M] net/bridge/bridge.ko
BTF [M] net/bluetooth/bluetooth.ko
LD [M] net/openvswitch/vport-geneve.ko
LD [M] net/openvswitch/vport-gre.ko
LD [M] net/nsh/nsh.ko
BTF [M] net/dns_resolver/dns_resolver.ko
BTF [M] net/sctp/sctp_diag.ko
BTF [M] net/sunrpc/auth_gss/auth_rpcgss.ko
BTF [M] net/nsh/nsh.ko
BTF [M] net/openvswitch/vport-geneve.ko
BTF [M] net/openvswitch/vport-gre.ko
BTF [M] net/openvswitch/vport-vxlan.ko
BTF [M] net/sunrpc/xprtrdma/rpcrdma.ko
LD [M] virt/lib/irqbypass.ko
BTF [M] net/sctp/sctp.ko
BTF [M] virt/lib/irqbypass.ko
BTF [M] net/sunrpc/sunrpc.ko
BTF [M] net/openvswitch/openvswitch.ko
BTF [M] net/ceph/libceph.ko
BTF [M] net/mac80211/mac80211.ko
vlad@DESKTOP-MKU600H:~/WSL2-Linux-Kernel$

```

Возможно, отсутствие /lib/modules станет проблемой в последующих заданиях с ядром Linux, но это получится узнать лишь в следующей работе.

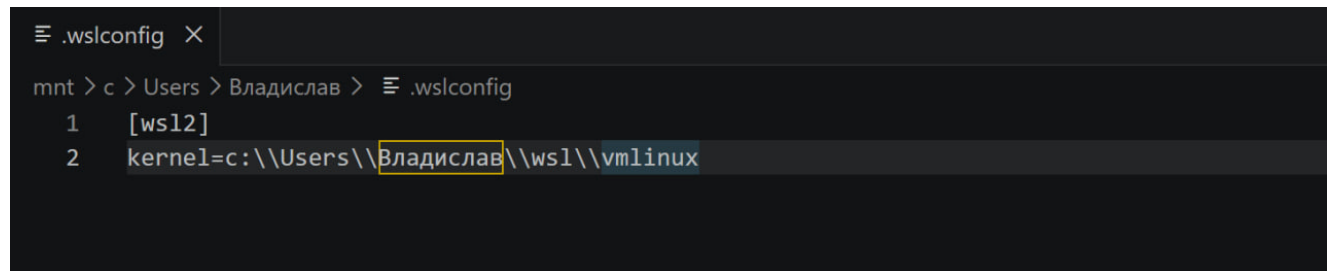
Дальше для wsl2 конфига нужно прописать путь. Поэтому создаем его в пользовательской части Windows 11 (оказалось, что это сэкономило много времени в будущем).

```
mkdir /mnt/c/Users/Владислав/wsl
cp vmlinux /mnt/c/Users/Владислав/wsl/
```

И открываем любым редактором .wslconfig
code /mnt/c/Users/Владислав/.wslconfig

Внутри .wslconfig пишем:

```
[wsl2]
kernel=c:\\Users\\Владислав\\wsl\\vmlinux
```

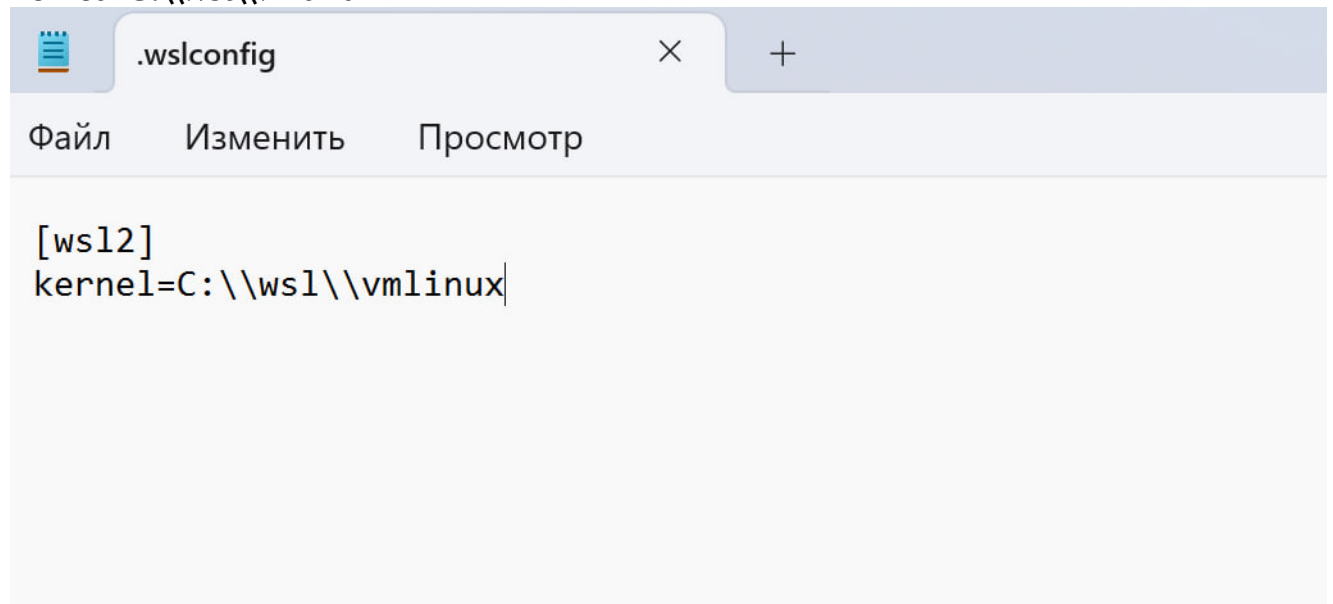


Где-то на этом этапе меня начинает беспокоить кириллица в пути, но ранее wsl с ней не имела проблем. Поэтому я продолжаю тестировать этот запуск.

Но все завершается с ошибкой при попытке запустить ядро. Приходится экстренно переносить vmlinux туда, где путь не будет содержать кириллицу. Вот на этом этапе хорошо, что под Windows можно было легко добраться в папку с .wslconfig

Поэтому прописываю:

```
[wsl2]
kernel=C:\\wsl\\vmlinux
```



И наконец-то, можно запустить ядро:

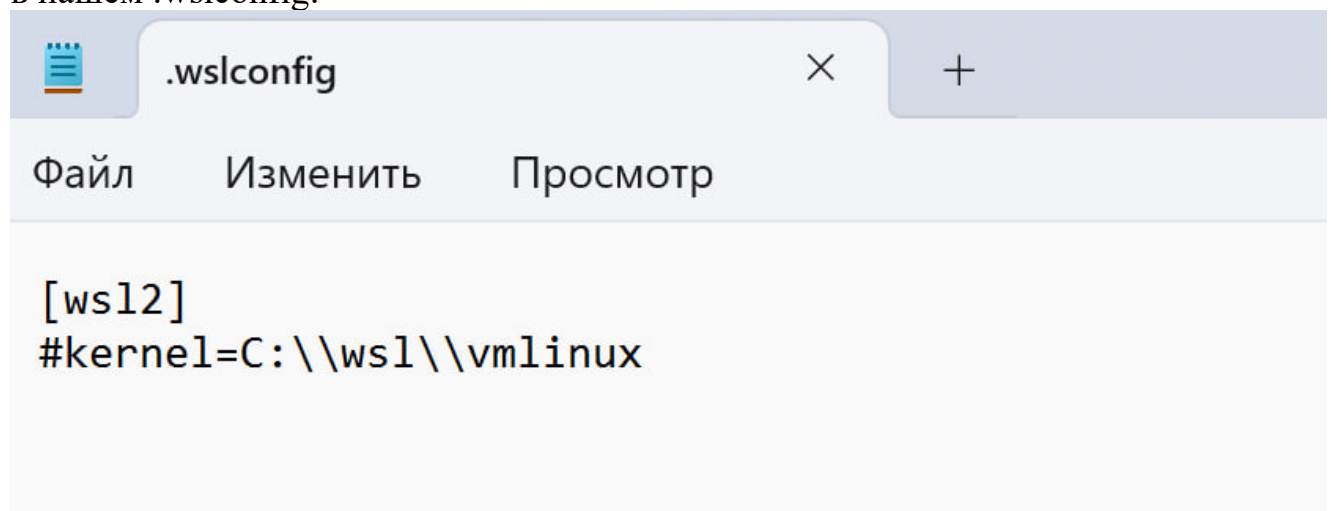
```
PS C:\Users\Владислав> wsl
Пользовательское ядро, указанное в C:\Users\Владислав\OneDrive\Рабочий стол\wsl\wslconfig, не найдено: "C:\wsl\wsl\vmlinux".
Код ошибки: Wsl/Service/CreateInstance/CreateVm/WSL_E_CUSTOM_KERNEL_NOT_FOUND
PS C:\Users\Владислав> wsl
vlad@DESKTOP-MKU600H: /mnt/c/Users/Владислав$ |
```

Проверяем текущую информацию о релизе:

```
vlad@DESKTOP-MKU600H:~$ uname -r
6.18.26.3-microsoft-standard-WSL2+
```

Если правильно понял, то значок "+" в конце названия ядра значит, что оно пользовательское, а не из исходного кода. На этом задача завершена и всё работает.

Чтобы не удалять новое ядро, и вернуться к нашей предыдущей сборке ядра (исходное от Microsoft), можно закомментировать путь к пользовательскому ядру в нашем .wslconfig:



После выхода из wsl и отключения командой `wsl --shutdown`, можно зайти в wsl и проверить, что старое ядро работает:

```
vlad@DESKTOP-MKU600H:~$ uname -r
6.6.87.2-microsoft-standard-WSL2
vlad@DESKTOP-MKU600H:~$ uname -a
Linux DESKTOP-MKU600H 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64
x86_64 GNU/Linux
vlad@DESKTOP-MKU600H:~$ |
```

Если интересно, то vmlinux весит 431 МБ.